

O que são algoritmos? Por que o estudo dos algoritmos vale a pena? Qual é o papel dos algoritmos em relação a outras tecnologias usadas em computadores? Neste capítulo, responderemos a essas perguntas.

1.1 ALGORITMOS

Informalmente, um *algoritmo* é qualquer procedimento computacional bem definido que toma algum valor ou conjunto de valores como *entrada* e produz algum valor ou conjunto de valores como *saída*. Portanto, um algoritmo é uma sequência de etapas computacionais que transformam a entrada na saída.

Também podemos considerar um algoritmo como uma ferramenta para resolver um *problema computacional* bem especificado. O enunciado do problema especifica em termos gerais a relação desejada entre entrada e saída. O algoritmo descreve um procedimento computacional específico para se conseguir essa relação entre entrada e saída.

Por exemplo, poderia ser necessário ordenar uma sequência de números em ordem não decrescente. Esse problema surge com frequência na prática e oferece um solo fértil para a apresentação de muitas técnicas de projeto e ferramentas de análise padronizadas. Vejamos como definir formalmente o *problema de ordenação*:

Entrada: Uma sequência de n números $\langle a_1, a_2, \dots, a_n \rangle$.

Saída: Uma permutação (reordenação) $\langle a_1', a_2', \dots, a_n' \rangle$ da sequência de entrada, tal que $a_1' \leq a_2' \leq \dots \leq a_n'$.

Por exemplo, dada a sequência de entrada $\langle 31, 41, 59, 26, 41, 58 \rangle$, um algoritmo de ordenação devolve como saída a sequência $\langle 26, 31, 41, 41, 58, 59 \rangle$. Tal sequência de entrada é denominada *instância* do problema de ordenação. Em geral, uma *instância de um problema* consiste na entrada (que satisfaz quaisquer restrições impostas no enunciado do problema) necessária para calcular uma solução para o problema.

Como muitos programas a utilizam como etapa intermediária, a ordenação é uma operação fundamental em ciência da computação e, por isso, há um grande número de bons algoritmos de ordenação à nossa disposição. O melhor algoritmo para determinada aplicação depende — entre outros fatores — do número de itens a ordenar, do grau de ordenação já apresentado por esses itens, das possíveis restrições aos valores dos itens, da arquitetura do computador e do tipo de dispositivo de armazenamento que será utilizado: memória principal, discos ou até mesmo fitas.

Diz-se que um algoritmo é *correto* se, para toda instância de entrada, ele parar com a saída correta. Dizemos que um algoritmo *resolve* o problema computacional dado. Um algoritmo incorreto poderia não parar em algumas instâncias de entrada ou poderia parar com uma resposta incorreta. Ao contrário do que se poderia esperar, às vezes os algoritmos incorretos podem ser úteis, se pudermos controlar sua taxa de erros. No Capítulo 31, veremos um exemplo de algoritmo com taxa de erro controlável quando estudarmos algoritmos para encontrar grandes números primos. Porém, de modo geral, nos concentraremos apenas em algoritmos corretos.

Um algoritmo pode ser especificado em linguagem comum como um programa de computador ou mesmo como um projeto de hardware. O único requisito é que a especificação deve fornecer uma descrição precisa do procedimento computacional a ser seguido.

Que tipos de problemas são resolvidos por algoritmos?

A ordenação não é de modo algum o único problema computacional para o qual os algoritmos foram desenvolvidos (e é provável que você já tenha suspeitado disso quando viu o tamanho deste livro.). As aplicações práticas de algoritmos estão por toda parte e incluem os exemplos a seguir:

- O Projeto Genoma Humano vem alcançando grande progresso no cumprimento de suas metas de identificar todos os 100.000 genes do DNA humano, determinar as sequências dos três bilhões de pares de bases químicas que constituem o DNA humano, armazenar essas informações em bancos de dados e desenvolver ferramentas para análise de dados. Cada uma dessas etapas exige algoritmos sofisticados. Embora as soluções para os vários problemas envolvidos estejam fora do escopo deste livro, muitos métodos aplicados à resolução desses problemas biológicos usam ideias apresentadas em vários capítulos deste livro, permitindo que os cientistas realizem tarefas e, ao mesmo tempo, utilizem os recursos com eficiência. As economias são de tempo, tanto humano quanto de máquina, e de dinheiro, já que mais informações podem ser extraídas de técnicas de laboratório.
- A Internet permite que pessoas em todo o mundo acessem e obtenham rapidamente grande quantidade de informações. Com o auxílio de algoritmos engenhosos, sites da Internet conseguem gerenciar e manipular esse grande volume de dados. Entre os exemplos de problemas que dependem essencialmente da utilização de algoritmos citamos a determinação de boas rotas para a transmissão de dados (técnicas para resolver tais problemas são apresentadas no Capítulo 24) e a utilização de um mecanismo de busca para encontrar rapidamente páginas em que estão determinadas informações (técnicas relacionadas são apresentadas nos Capítulos 11 e 32).
- O comércio eletrônico permite que mercadorias e serviços sejam negociados e trocados eletronicamente e depende do sigilo de informações pessoais, como números de cartões de crédito, senhas e extratos bancários. Entre as principais tecnologias utilizadas no comércio eletrônico estão a criptografia de chave pública e as assinaturas digitais (estudadas no Capítulo 31), ambas baseadas em algoritmos numéricos e na teoria dos números.
- Na indústria e em outros empreendimentos comerciais, muitas vezes é preciso alocar recursos escassos da maneira mais benéfica possível. Uma empresa petrolífera talvez deseje saber onde localizar seus poços para maximizar o lucro esperado. Um político talvez queira determinar onde gastar dinheiro em publicidade de campanha para maximizar as chances de vencer uma eleição. Uma empresa de transporte aéreo poderia querer designar tripulações a voos da forma menos dispendiosa possível, garantindo que cada voo seja atendido e que as regulamentações do governo relativas à escala das tripulações sejam obedecidas. Um provedor de serviços da Internet talvez queira definir onde alocar recursos adicionais para servir a seus clientes com mais eficiência. Todos esses são exemplos de problemas que podem ser resolvidos com a utilização de programação linear, que estudaremos no Capítulo 29.

Embora alguns dos detalhes desses exemplos estejam fora do escopo deste livro, forneceremos técnicas básicas que se aplicam a esses problemas e áreas de problemas. Também mostraremos como resolver muitos problemas específicos, inclusive os seguintes:

- Temos um mapa rodoviário no qual estão marcadas as distâncias entre cada par de interseções adjacentes e queremos determinar a rota mais curta entre uma interseção e outra. O número de rotas possíveis pode ser enorme, ainda que sejam descartadas as rotas que se entrecruzam. Como escolher a mais curta de todas as rotas possíveis? Aqui, modelamos o mapa rodoviário (que é ele próprio um modelo das estradas reais) como um grafo (o que veremos na Parte VI e no Apêndice B) e desejamos determinar o caminho mais curto de um vértice até outro no grafo. Veremos como resolver esse problema com eficiência no Capítulo 24.
- Temos duas sequências ordenadas de símbolos, $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, e queremos determinar uma subsequência comum mais longa de X e Y . Uma subsequência de X é apenas X com alguns (ou, possivelmente, todos ou nenhum) de seus elementos removidos. Por exemplo, uma subsequência de $\langle A, B, C, D, E, F, G \rangle$ seria $\langle B, C, E, G \rangle$. O comprimento de uma subsequência comum mais longa de X e Y nos dá uma ideia do grau de semelhança dessas duas sequências. Por exemplo, se as duas sequências forem pares de base em

filamentos de DNA, poderemos considerá-las semelhantes se tiverem uma subsequência comum longa. Se X tiver m símbolos e Y tiver n símbolos, então X e Y terão 2^m e 2^n possíveis subsequências, respectivamente. Selecionar todas as possíveis subsequências de X e Y e então combiná-las poderá tomar um tempo proibitivamente longo, a menos que m e n sejam muito pequenos. No Capítulo 15 veremos como usar uma técnica geral conhecida como programação dinâmica para resolver esse problema com eficiência muito maior.

- Temos um projeto mecânico apresentado como um catálogo de peças no qual cada uma pode incluir instâncias de outras peças e precisamos organizar uma lista ordenada de peças de modo que cada uma apareça antes de qualquer peça que a utilize. Se o projeto compreender n peças, então haverá $n!$ ordenações possíveis, onde $n!$ denota a função fatorial. Como a função fatorial cresce ainda mais rapidamente do que uma função exponencial, não existe uma possibilidade viável de gerar cada ordenação possível e então verificar se, dentro daquela ordenação, cada peça aparece antes das peças que a utilizam (a menos que tenhamos apenas um pequeno número de peças). Esse problema é uma instância de ordenação topológica, e estudaremos como resolvê-lo com eficiência no Capítulo 22.
- Temos n pontos no plano e desejamos determinar a envoltória convexa desses pontos. A envoltória convexa é o menor polígono convexo que contém os pontos. Intuitivamente, podemos imaginar que cada ponto seja representado pela cabeça saliente de um prego fixado a uma tábua. A envoltória convexa seria representada por um elástico apertado que contorna todos os pregos. Cada prego pelo qual o elástico passar é um vértice da envoltória convexa (veja um exemplo na Figura 33.6). Qualquer um dos 2^n subconjuntos dos pontos poderia ser os vértices da envoltória convexa. Porém, saber quais pontos são vértices da envoltória convexa não é suficiente, já que também precisamos conhecer a ordem em que eles aparecem. Portanto, há muitas escolhas para os vértices da envoltória convexa. O Capítulo 33 apresenta dois bons métodos para determinar a envoltória convexa.

Essas listas estão longe de esgotar os exemplos (como é provável que você já tenha imaginado de novo pelo peso deste livro), mas exibem duas características comuns a muitos problemas algorítmicos interessantes:

1. Eles têm muitas soluções candidatas, a grande maioria das quais não resolve o problema que temos em mãos. Encontrar uma solução que o resolva, ou uma solução que seja a “melhor”, pode representar um desafio significativo.
2. Eles têm aplicações práticas. Dentre os problemas da lista que apresentamos, o da determinação do caminho mais curto é o que fornece os exemplos mais fáceis. Uma empresa de transporte rodoviário ou ferroviário tem interesse financeiro em determinar os caminhos mais curtos em uma rede rodoviária ou ferroviária porque percursos menores resultam em menores custos de mão de obra e combustível. Ou um nó de roteamento na Internet pode precisar encontrar o caminho mais curto através da rede, para rotear uma mensagem com rapidez. Ou pode ser que alguém que deseje viajar de carro de Nova York a Boston queira encontrar instruções em um site Web adequado ou usar seu GPS durante o percurso.

Nem todo problema resolvido por algoritmos tem um conjunto de soluções candidatas fáceis de identificar. Por exemplo, suponha que temos um conjunto de valores numéricos que representam amostras de um sinal e que queremos calcular a transformada discreta de Fourier dessas amostras. A transformada discreta de Fourier converte o domínio do tempo para o domínio da frequência, produzindo um conjunto de coeficientes numéricos, de modo que podemos determinar a força de várias frequências no sinal amostrado. Além de estarem no cerne do processamento de sinais, as transformadas discretas de Fourier também se aplicam à compressão de dados e à multiplicação de grandes polinômios e inteiros. O Capítulo 30 apresenta um algoritmo eficiente para esse problema, a transformada rápida de Fourier (comumente denominada FFT), e também apresenta o esquema de projeto de um circuito de *hardware* para calcular a FFT.

Estruturas de dados

Este livro também contém várias estruturas de dados. Uma *estrutura de dados* é um modo de armazenar e organizar dados com o objetivo de facilitar acesso e modificações. Nenhuma estrutura de dados única funciona bem para todas as finalidades e, por isso, é importante conhecer os pontos fortes e as limitações de várias delas.

Técnica

Embora você possa usar este livro como um “livro de receitas” para algoritmos, é possível que algum dia encontre um problema cujo algoritmo publicado (muitos dos exercícios e problemas deste livro, por exemplo) não consiga achar imediatamente. Este livro lhe ensinará técnicas de projeto e análise de algoritmos para que você possa desenvolver algoritmos por conta própria, mostrar que eles fornecem a resposta correta e entender sua eficiência.

Capítulos diferentes abordam aspectos diferentes da solução de problemas de algoritmos. Alguns abordam problemas específicos, como determinar medianas e ordenar dados estatísticos, no Capítulo 9, calcular árvores geradoras (*spanning trees*) mínimas, no Capítulo 23, e determinar um fluxo máximo em uma rede, no Capítulo 26. Outros capítulos abordam técnicas como a de divisão e conquista, no Capítulo 4, programação dinâmica, no Capítulo 15, e análise amortizada, no Capítulo 17.

Problemas difíceis

A maior parte deste livro trata de algoritmos eficientes. Nossa medida habitual de eficiência é a velocidade, isto é, quanto tempo um algoritmo demora para produzir seu resultado. Porém, existem alguns problemas para os quais não se conhece nenhuma solução eficiente. O Capítulo 34 estuda um subconjunto interessante desses problemas, conhecidos como NP-completos.

Por que os problemas NP-completos são interessantes? Em primeiro lugar, embora nenhum algoritmo eficiente para um problema NP-completo tenha sido encontrado até agora, ninguém jamais provou que não é possível existir um algoritmo eficiente para tal problema. Em outras palavras, ninguém sabe se existem ou não algoritmos eficientes para problemas NP-completos. Em segundo lugar, o conjunto de problemas NP-completos tem a propriedade notável de que, se existir um algoritmo eficiente para qualquer um deles, então existem algoritmos eficientes para todos eles. Essa relação entre os problemas NP-completos torna a falta de soluções eficientes ainda mais torturante. Em terceiro lugar, vários problemas NP-completos são semelhantes mas não idênticos a problemas para os quais sabemos existir algoritmos eficientes. Cientistas da computação ficam intrigados com o fato de que uma pequena mudança no enunciado do problema pode provocar uma grande alteração na eficiência do melhor algoritmo conhecido.

É bom que você conheça os problemas NP-completos porque alguns deles surgem com frequência surpreendente em aplicações reais. Se você tiver de produzir um algoritmo eficiente para um problema NP-completo, é provável que perca muito tempo em uma busca infrutífera. Por outro lado, se você conseguir mostrar que o problema é NP-completo, poderá dedicar seu tempo ao desenvolvimento de um algoritmo eficiente que ofereça uma solução boa, embora não a melhor possível.

Como exemplo concreto, considere uma empresa transportadora que tenha um depósito central. Todo dia, cada caminhão é carregado no depósito e enviado a diversos locais para fazer entregas. No final do dia, cada caminhão tem de estar de volta ao depósito para ser preparado para a carga no dia seguinte. Para reduzir custos, a empresa quer selecionar uma ordem de pontos de entrega que represente a menor distância total a ser percorrida por cada caminhão. Esse problema é o famoso “problema do caixeiro-viajante”, e é NP-completo — não tem nenhum algoritmo eficiente conhecido. Contudo, adotando-se certas premissas, há algoritmos eficientes que fornecem uma distância total não muito acima da menor possível. O Capítulo 35 discute esses “algoritmos de aproximação”.

Paralelismo

Durante muitos anos pudemos contar com uma taxa regular de aumento da velocidade de relógio dos processadores. Porém, limitações físicas representam um entrave fundamental ao crescimento constante dessas

velocidades: como a densidade de potência aumenta superlinearmente com a velocidade de relógio, existe o risco de derretimento dos chips quando essas velocidades atingem um certo nível. Portanto, para executar mais cálculos por segundo, o projeto moderno de chips prevê não apenas um, mas vários “núcleos” de processamento. Podemos comparar esses computadores com vários núcleos a vários computadores sequenciais em um único chip; em outras palavras, eles são um tipo de “computador paralelo”. Para obter o melhor desempenho desses processadores com vários núcleos, precisamos produzir algoritmos que considerem o paralelismo. O Capítulo 27 apresenta um modelo de algoritmo “multithread” que tira proveito de núcleos múltiplos. Do ponto de vista teórico, esse modelo é vantajoso e constitui a base de vários modelos eficientes de programas de computador, entre eles um programa para campeonatos de xadrez.

Exercícios

- 1.1-1 Cite um exemplo real que exija ordenação ou um exemplo real que exija o cálculo de uma envoltória convexa.
- 1.1-2 Além da velocidade, que outras medidas de eficiência poderiam ser usadas em uma configuração real?
- 1.1-3 Selecione uma estrutura de dados que você já tenha visto antes e discuta seus pontos fortes e suas limitações.
- 1.1-4 Em que aspectos os problemas anteriores do caminho mais curto e do caixeiro-viajante são semelhantes? Em que aspectos eles são diferentes?
- 1.1-5 Mostre um problema real no qual apenas a melhor solução servirá. Em seguida, apresente um problema em que baste uma solução que seja “aproximadamente” a melhor.

1.2 ALGORITMOS COMO TECNOLOGIA

Suponha que os computadores fossem infinitamente rápidos e que a memória do computador fosse gratuita. Você teria alguma razão para estudar algoritmos? A resposta é sim, ainda que apenas porque você gostaria de demonstrar que seu método de solução termina, e o faz com a resposta correta.

Se os computadores fossem infinitamente rápidos, qualquer método correto para resolver um problema serviria. É provável que você quisesse que sua implementação estivesse dentro dos limites da boa prática de engenharia de software (isto é, que ela fosse bem documentada e projetada) mas, na maior parte das vezes, você utilizaria o método que fosse mais fácil de implementar.

É claro que os computadores podem ser rápidos, mas eles não são infinitamente rápidos. A memória pode ser de baixo custo, mas não é gratuita. Assim, o tempo de computação é um recurso limitado, bem como o espaço na memória. Esses recursos devem ser usados com sensatez, e algoritmos eficientes em termos de tempo ou espaço o ajudarão a usá-los assim.

Eficiência

Algoritmos diferentes criados para resolver o mesmo problema muitas vezes são muito diferentes em termos de eficiência. Essas diferenças podem ser muito mais significativas que as diferenças relativas a hardware e software.

Como exemplo, no Capítulo 2 estudaremos dois algoritmos de ordenação. O primeiro, conhecido como **ordenação por inserção**, leva um tempo aproximadamente igual a $c_1 n^2$ para ordenar n itens, onde c_1 é uma constante que não depende de n . Isto é, ele demora um tempo aproximadamente proporcional a n^2 . O segundo, de **ordenação por intercalação**, leva um tempo aproximadamente igual a $c_2 n \lg n$, onde $\lg n$ representa $\log_2 n$ e c_2 é outra constante que também não depende de n . A ordenação por inserção normalmente tem um fator constante menor que a ordenação

por intercalação; assim, $c_1 < c_2$. Veremos que os fatores constantes podem causar um impacto muito menor sobre o tempo de execução que a dependência do tamanho da entrada n . Se representarmos o tempo de execução da ordenação por inserção por $c_1 n \cdot n$ e o tempo de execução da ordenação por intercalação por $c_2 n \cdot \lg n$, veremos que, enquanto o fator do tempo de execução da ordenação por inserção é n , o da ordenação por intercalação é $\lg n$, que é muito menor (por exemplo, quando $n = 1.000$, $\lg n$ é aproximadamente 10 e quando n é igual a um milhão, $\lg n$ é, aproximadamente, só 20). Embora a ordenação por inserção em geral seja mais rápida que a ordenação por intercalação para pequenos tamanhos de entradas, tão logo o tamanho da entrada n se torne grande o suficiente a vantagem da ordenação por intercalação de $\lg n$ contra n compensará com sobras a diferença em fatores constantes. Independentemente do quanto c_1 seja menor que c_2 , sempre haverá um ponto de corte além do qual a ordenação por intercalação será mais rápida.

Como exemplo concreto, vamos comparar um computador mais rápido (computador A) que executa a ordenação por inserção com um computador mais lento (computador B) que executa a ordenação por intercalação. Cada um deve ordenar um arranjo de dez milhões de números. (Embora 10 milhões de números possa parecer muito, se os números forem inteiros de oito bytes, a entrada ocupará cerca de 80 megabytes e caberá com grande folga até mesmo na memória de um laptop barato.) Suponha que o computador A execute dez bilhões de instruções por segundo (mais rapidamente do que qualquer computador sequencial existente na época da redação deste livro) e que o computador B execute apenas dez milhões de instruções por segundo; assim, o computador A será 1.000 vezes mais rápido que o computador B em capacidade bruta de computação. Para tornar a diferença ainda mais drástica, suponha que o programador mais astucioso do mundo codifique a ordenação por inserção em linguagem de máquina para o computador A e que o código resultante exija $2n^2$ instruções para ordenar n números. Suponha ainda que um programador médio implemente a ordenação por intercalação utilizando uma linguagem de alto nível com um compilador ineficiente, sendo que o código resultante totaliza $50n \lg n$ instruções. Para ordenar 10 milhões de números, o computador A leva

$$\frac{2 \cdot (10^7)^2 \text{ instruções}}{10^{10} \text{ instruções/segundo}} = 20.000 \text{ segundos (mais de 5,5 horas);}$$

por outro lado, o computador B leva

$$\frac{50 \cdot 10^7 \lg 10^7 \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 1163 \text{ segundos (menos de 20 minutos).}$$

Usando um algoritmo cujo tempo de execução cresce mais lentamente, até mesmo com um compilador fraco, o computador B funciona mais de 17 vezes mais rapidamente que o computador A! A vantagem da ordenação por intercalação é ainda mais evidente quando ordenamos 100 milhões de números: onde a ordenação por inserção demora mais de 23 dias, a ordenação por intercalação demora menos de quatro horas. Em geral, à medida que o tamanho do problema aumenta, também aumenta a vantagem relativa da ordenação por intercalação.

Algoritmos e outras tecnologias

O exemplo anterior mostra que os algoritmos, como o hardware de computadores, devem ser considerados como uma **tecnologia**. O desempenho total do sistema depende da escolha de algoritmos eficientes tanto quanto da escolha de hardware rápido. Os rápidos avanços que estão ocorrendo em outras tecnologias computacionais também estão sendo observados em algoritmos. Você poderia questionar se os algoritmos são verdadeiramente tão importantes para os computadores contemporâneos levando em consideração outras tecnologias avançadas, como:

- arquiteturas computacionais e tecnologias de fabricação avançadas;
- interfaces gráficas de usuário (GUIs) intuitivas e fáceis de usar;
- sistemas orientados a objetos;

- tecnologias integradas da Web e
- redes de alta velocidade, com fio e sem fio.

A resposta é: sim. Embora algumas aplicações não exijam explicitamente conteúdo algorítmico no nível da aplicação (como algumas aplicações simples baseadas na Web), a maioria exige. Por exemplo, considere um serviço da Web que determina como viajar de um local para outro. Sua implementação dependeria de hardware rápido, de uma interface gráfica de usuário, de redes remotas, além de, possivelmente, orientação a objetos. Contudo, também exigiria algoritmos para certas operações, como descobrir rotas (talvez empregando um algoritmo de caminho mais curto), apresentar mapas e interpolar endereços.

Além disso, até mesmo uma aplicação que não exija conteúdo algorítmico no nível da aplicação depende muito de algoritmos. A aplicação depende de hardware rápido? O projeto de hardware utilizou algoritmos. A aplicação depende de interfaces gráficas de usuário? O projeto de qualquer GUI depende de algoritmos. A aplicação depende de rede? O roteamento em redes depende muito de algoritmos. A aplicação foi escrita em uma linguagem diferente do código de máquina? Então, ela foi processada por um compilador, um interpretador ou um montador, e todos fazem uso extensivo de algoritmos. Os algoritmos estão no núcleo da maioria das tecnologias usadas em computadores contemporâneos.

Além disso, com a capacidade cada vez maior dos computadores, nós os utilizamos mais do que nunca para resolver problemas cada vez maiores. Como vimos na comparação anterior entre ordenação por inserção e ordenação por intercalação, é nos problemas maiores que as diferenças entre a eficiência dos algoritmos se tornam particularmente notáveis.

Uma sólida base de conhecimento e técnica de algoritmos é uma das características que separam os programadores verdadeiramente qualificados dos novatos. Com a moderna tecnologia de computação, você pode executar algumas tarefas sem saber muito sobre algoritmos; porém, com uma boa base em algoritmos, é possível fazer muito, muito mais.

Exercícios

- 1.2-1** Cite um exemplo de aplicação que exige conteúdo algorítmico no nível da aplicação e discuta a função dos algoritmos envolvidos.
- 1.2-2** Suponha que estamos comparando implementações de ordenação por inserção e ordenação por intercalação na mesma máquina. Para entradas de tamanho n , a ordenação por inserção é executada em $8n^2$ passos, enquanto a ordenação por intercalação é executada em $64n \lg n$ passos. Para quais valores de n a ordenação por inserção supera a ordenação por intercalação?
- 1.2-3** Qual é o menor valor de n tal que um algoritmo cujo tempo de execução é $100n^2$ funciona mais rapidamente que um algoritmo cujo tempo de execução é $2n$ na mesma máquina?

Problemas

1-1 Comparação entre tempos de execução

Para cada função $f(n)$ e cada tempo t na tabela a seguir, determine o maior tamanho n de um problema que pode ser resolvido no tempo t , considerando que o algoritmo para resolver o problema demore $f(n)$ microssegundos.

	1 segundo	1 minuto	1 hora	1 dia	1 mês	1 ano	1 século
$\lg n$							

$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							

NOTAS DO CAPÍTULO

Existem muitos textos excelentes sobre o tópico geral de algoritmos, entre eles os de Aho, Hopcroft e Ullman [5, 6]; Baase e Van Gelder [28]; Brassard e Bratley [54]; Dasgupta, Papadimitriou e Vazirani [82]; Goodrich e Tamassia [148]; Hofri [175]; Horowitz, Sahni e Rajasekaran [181]; Johnsonbaugh e Schaefer [193]; Kingston [205]; Kleinberg e Tardos [208]; Knuth [209, 210, 211]; Kozen [220]; Levitin [235]; Manber [242]; Mehlhorn [249, 250, 251]; Purdom e Brown [287]; Reingold, Nievergelt e Deo [293]; Sedgewick [306]; Sedgewick e Flajolet [307]; Skiena [318] e Wilf [356]. Alguns dos aspectos mais práticos do projeto de algoritmos são discutidos por Bentley [42, 43] e Gonnet [145]. Pesquisas na área de algoritmos também podem ser encontradas no *Handbook of Theoretical Computer Science, Volume A* [342] e no *CRC Algorithms and Theory of Computation Handbook* [25]. Avaliações de algoritmos usados em biologia computacional podem ser encontrados nos livros didáticos de Gusfield [156], Pevzner [275], Setubal e Meidanis [310] e Waterman [350].